General

1. **Any violation to the guide is allowed if it enhances readability.**

The main goal of the recommendation is to improve readability and thereby the understanding and the maintainability and general quality of the code. It is impossible to cover all the specific cases in a general guide and the programmer should be flexible.

Naming

2. **Names representing packages should be in all lower case.**

```
mypackage, com.company.application.ui
```

Package naming convention used in the Java core packages. The initial package name representing the domain name must be in lower case.

3. **Names representing types must be nouns and written in mixed case starting with upper case.**

```
Line, AudioSystem
```

Common practice in the Java development community and also the type naming convention used in the Java core packages.

4. **Variable names must be in mixed case starting with lower case.**

```
line, audioSystem
```

Common practice in the Java development community and also the naming convention for variables used in the Java core packages. Makes variables easy to distinguish from types, and effectively resolves potential naming collision as in the declaration

```
Line line;
```

5. **Names representing constants (final variables) must be all uppercase using underscore to separate words.**

```
MAX_ITERATIONS, COLOR_RED
```

Common practice in the Java development community and also the naming convention used by the Java core packages.
In general, the use of such constants should be minimized.

6. **Names representing methods must contain a verb and written in mixed case starting with lower case.**

```
getName(), computeTotalWidth()
```

Common practice in the Java development community and also the naming convention used in the Java core packages. This is identical to variable names, but methods in Java are already distinguishable from variables by their specific form.

7. **Abbreviations and acronyms should not be uppercase when used as name.**

```
exportHtmlSource(); // NOT: exportHTMLSource();
readJsonFile();     // NOT: readJSONFile();
```

Using all uppercase for the base name will give conflicts with the naming conventions given above. A variable of this type would have to be named `jSON`, `hTML` etc. which obviously is not very readable. Another problem is illustrated in the examples above: When the name is connected to another, the readability is seriously reduced. The word following the acronym does not stand out as it should.

8. **Private class members should have underscore suffix.**

```
class Person
{
  private String name_;

  :
}
```

Apart from its name and its type, the *scope* of a member is its most important feature. Indicating class scope by using underscore makes it easy to distinguish class members from local scratch variables. This is important because members are considered more significant than method arguments and variables, and should be treated with special care by the programmer.

A side effect of the underscore naming convention is that it nicely resolves the problem of finding reasonable variable names for setter methods:

```
void setName(String name)
{
  name_ = name;
}
```

An issue is whether the underscore should be added as a *prefix* or as a *suffix*. Both practices are commonly used, but the latter is recommended because it seem to best preserve the readability of the name.

It should be noted that scope identification in variables has been controversial. The practice has however gained widespread acceptance in the professional development community.

9. **Generic variables should have the same name as their type.**

```
void setTopic(Topic topic) // NOT: void setTopic(Topic value)
                           // NOT: void setTopic(Topic aTopic)
                           // NOT: void setTopic(Topic t)

void connect(Database database) // NOT: void connect(Database db)
                                // NOT: void connect(Database postgresDB)
```

Reduce complexity by reducing the number of terms and names used. Also makes it easy to deduce the type given a variable name only. If for some reason this convention doesn't seem to *fit* it is a strong indication that the type name is badly chosen.

Non-generic variables have a *role*. These variables can often be named by combining role and type:

```
Point  startingPoint, centerPoint;
Name   loginName;
```

10. **All names should be written in English.**

Using English for names simplifies global collaboration, ensure consistency between systems, simplify onboarding process, ensure maintainability and maintains technical jargon.

11. **Variables with a large scope should have long names, variables with a small scope can have short names.**

Scratch variables used for temporary storage or indices are best kept short. A programmer reading such variables should be able to assume that its value is not used outside a few lines of code. Common scratch variables for integers are *i*, *j*, *k*, *m*, *n* and for characters *c* and *d*.

12. **The name of the object is implicit, and should be avoided in a method name.**

```
line.getLength();   // NOT: line.getLineLength();
```

The latter might seem natural in the class declaration, but proves superfluous in use, as shown in the example.

13. **The terms *get/set* must be used where an attribute is accessed directly.**

```
employee.getName();
employee.setName(name);

matrix.getElement(2, 4);
matrix.setElement(2, 4, value);
```

Common practice in the Java community and the convention used in the Java core packages.

14. ***is*-prefix should be used for boolean variables and methods.**

```
isSet, isVisible, isFinished, isFound, isOpen
```

This is the naming convention for boolean methods and variables used in the Java core packages. Using the *is*-prefix solves a common problem of choosing bad boolean names like *status* or *flag*. `isStatus` or `isFlag` simply doesn't fit, and the programmer is forced to chose more meaningful names.

Setter methods for boolean variables must have *set*-prefix as in:

```
void setFound(boolean isFound);
```

There are a few alternatives to the *is*-prefix that fits better in some situations. These are *has*-, *can*- and *should*- prefixes:

```
boolean hasLicense();
boolean canEvaluate();
boolean shouldAbort = false;
```

15. **The term *compute* can be used in methods where something is computed.**

```
valueSet.computeAverage();
matrix.computeInverse()
```

Give the reader the immediate clue that this is a potential time consuming operation, and if used repeatedly, he might consider caching the result. Consistent use of the term enhances readability.

16. **The term *find* can be used in methods where something is looked up.**

```
vertex.findNearestVertex();
matrix.findSmallestElement();
node.findShortestPath(Node destinationNode);
```

Give the reader the immediate clue that this is a simple look up method with a minimum of computations involved. Consistent use of the term enhances readability.

17. **The term *initialize* can be used where an object or a concept is established.**

```
document.initializeFontSet();
```

The American *initialize* should be preferred over the English *initialise*. Abbreviation *init* must be avoided.

18. **User interface components names should be suffixed by the element type.**

```
widthScale, nameTextField, leftScrollbar, mainPanel, fileToggle, minLabel, printerD
```

Enhances readability since the name gives the user an immediate clue of the type of the variable and thereby the available resources of the object.

19. **Plural form should be used on names representing a collection of objects.**

```
Collection<Point> points;
int[] values;
```

Enhances readability since the name gives the user an immediate clue of the type of the variable and the operations that can be performed on its elements.

20. ***n*-prefix should be used for variables representing a number of objects.**

```
nPoints, nTables
```

The notation is taken from mathematics where it is an established convention for indicating a number of objects.

Note that *num* prefix is used in the core Java packages for these variables. This is probably meant as an abbreviation of *number of*, but as it looks more like *number* and it makes the variable name strange and misleading. If "number of" is the preferred phrase, *numberOf* prefix can be used instead of just *n*. *num* prefix must not be used.

21. ***No* suffix should be used for variables representing an entity number.**

```
tableNo, employeeNo
```

The notation is taken from mathematics where it is an established convention for indicating an entity number.

An elegant alternative is to prefix such variables with an *i*: `iTable`, `iEmployee`. This effectively makes them named iterators.

22. **Iterator variables should be called *i*, *j*, *k* etc.**

```
for (Iterator i = points.iterator(); i.hasNext()) {
  :
}

for (int i = 0; i < nTables; i++) {
  :
}
```

The notation is taken from mathematics where it is an established convention for indicating iterators.

Variables named *j*, *k* etc. should be used for nested loops only.

23. **Complement names must be used for complement entities.**

```
get/set, add/remove, create/destroy, start/stop, insert/delete,
increment/decrement, old/new, begin/end, first/last, up/down, min/max,
next/previous, open/close, show/hide, suspend/resume, etc.
```

Reduce complexity by symmetry.

24. **Abbreviations in names should be avoided.**

```
computeAverage();               // NOT: compAvg();
ActionEvent event;              // NOT: ActionEvent e;
catch (Exception exception) {   // NOT: catch (Exception e) {
```

There are two types of words to consider. First are the common words listed in a language dictionary. These must never be abbreviated. Never write:

`cmd`	instead of	`command`
`comp`	instead of	`compute`
`cp`	instead of	`copy`
`e`	instead of	`exception`
`init`	instead of	`initialize`
`pt`	instead of	`point`

etc.

Then there are domain specific phrases that are more naturally known through their acronym or abbreviations. These phrases should be kept abbreviated. Never write:

`HypertextMarkupLanguage`	instead of	`html`
`CentralProcessingUnit`	instead of	`cpu`
`PriceEarningRatio`	instead of	`pe`

etc.

25. **Negated boolean variable names must be avoided.**

```
bool isError; // NOT: isNoError
bool isFound; // NOT: isNotFound
```

The problem arise when the logical not operator is used and double negative arises. It is not immediately apparent what `!isNotError` means.

26. **Associated constants (final variables) should be prefixed by a common type name.**

```
final int COLOR_RED   = 1;
final int COLOR_GREEN = 2;
final int COLOR_BLUE  = 3;
```

This indicates that the constants belong together, and what concept the constants represents. An alternative to this approach is to put the constants inside an interface effectively prefixing their names with the name of the interface:

```
interface Color
{
  int RED   = 1;
  int GREEN = 2;
  int BLUE  = 3;
}
```

27. **Exception classes should be suffixed with *Exception*.**

```
class AccessException extends Exception
{
  :
}
```

Exception classes are really not part of the main design of the program, and naming them like this makes them stand out relative to the other classes. This standard is used in the core Java packages.

28. **Default interface implementations can be prefixed by *Default*.**

```
class DefaultTableCellRenderer
  implements TableCellRenderer
{
  :
}
```

It is not uncommon to create a simplistic class implementation of an interface providing default behavior to the interface methods. The convention of prefixing these classes by *Default* has been adopted by the core Java packages.

29. **Singleton classes should return their sole instance through method *getInstance*.**

```
public final class UnitManager
{
  private final static UnitManager instance_ = new UnitManager();

  private UnitManager()
  {
    :
  }

  public static UnitManager getInstance()  // NOT: get() or instance() or unitManag
  {
    return instance_;
  }
}
```

Common practice in the Java community though not consistently followed by the core Java packages. The above layout is the preferred pattern.

30. **Classes that creates instances on behalf of others (*factories*) can do so through method *new[ClassName]***

```
public class PointFactory
{
  public Point newPoint(...)
  {
    ...
  }
}
```

Indicates that the instance is created by *new* inside the factory method and that the construct is a controlled replacement of `new Point()`.

31. **Functions (methods returning an object) should be named after what they return and procedures (*void* methods) after what they do.**

Increase readability. Makes it clear what the unit should do and especially all the things it is *not* supposed to do. This again makes it easier to keep the code clean of side effects.

Files

32. **Java source files should have the extension *.java*.**

```
Point.java
```

Enforced by the Java tools.

33. **Classes should be declared in individual files with the file name matching the class name. Secondary private classes can be declared as inner classes and reside in the file of the class they belong to.**

Enforced by the Java tools.

34. **File content can be kept within 80 columns.**

This convention has its roots in older systems, but it still offers some practical benefits like improved readability, avoid horizontal scrilling, consistency across monitors and simplified code reviews.

35. **Special characters like TAB and page break must be avoided.**

Such characters may potentially cause inconsistent display across systems, give code alignment issues, reduce readability and give version control conflicts.

36. **The incompleteness of split lines should be made obvious.**

```
totalSum = a + b + c +
           d + e;

method(param1, param2,
       param3);

setText ("Long line split" +
         "into two parts.");

for (int tableNo = 0; tableNo < nTables;
     tableNo += tableStep) {
  ...
}
```

Split lines occurs when a statement exceed the 80 column limit given above. It is difficult to give rigid rules for how lines should be split, but the examples above should give a general hint.

In general:

- Break after a comma.
- Break after an operator.
- Align the new line with the beginning of the expression on the previous line.

Statements

37. **The `package` statement must be the first statement of the file. All files should belong to a specific package.**

The `package` statement location is enforced by the Java language. Letting all files belong to an actual (rather than the Java default) package enforces Java language object oriented programming techniques.

38. **The `import` statements must follow the `package` statement. `import` statements should be sorted with the most fundamental packages first, and grouped with associated packages together and one blank line between groups.**

```
import java.io.IOException;
import java.net.URL;

import java.rmi.RmiServer;
import java.rmi.server.Server;

import javax.swing.JPanel;
import javax.swing.event.ActionEvent;

import org.linux.apache.server.SoapServer;
```

The `import` statement location is enforced by the Java language. The sorting makes it simple to browse the list when there are many imports, and it makes it easy to determine the dependencies of the present package The grouping reduce complexity by collapsing related information into a common unit.

39. **Imported classes must always be listed explicitly.**

```
import java.util.List; // NOT: import java.util.*;
import java.util.ArrayList;
import java.util.HashSet;
```

Importing classes explicitly gives an excellent documentation value for the class at hand and makes the class easier to comprehend and maintain.

Appropriate tools should be used in order to always keep the import list minimal and up to date.

Declarations

40. **Class and Interface declarations should be organized in the following manner:**
    1. **Class/Interface documentation.**
    2. **`class` or `interface` statement.**
    3. **Class (`static`) variables in the order `public`, `protected`, package (no access modifier), `private`.**
    4. **Instance variables in the order `public`, `protected`, package (no access modifier), `private`.**
    5. **Constructors.**
    6. **Methods (no specific order).**

Reduce complexity by making the location of each class element predictable.

41. **Method modifiers should be given in the following order:**
<access> static abstract synchronized <unusual> final native
The *<access>* modifier (if present) must be the first modifier.

```
public static double square(double a)  // NOT: static public double square(double a
```

<access> is one of *public*, *protected* or *private* while *<unusual>* includes *volatile* and *transient*.

The most important lesson here is to keep the *access* modifier as the first modifier. Of the possible modifiers, this is by far the most important, and it must stand out in the method declaration. For the other modifiers, the order is less important, but it make sense to have a fixed convention.

Variables

42. **Variables should be initialized where they are declared and they should be declared in the smallest scope possible.**

This ensures that variables are valid at any time. Sometimes it is impossible to initialize a variable to a valid value where it is declared. In these cases it should be left uninitialized rather than initialized to some phony value.

43. **Variables must never have dual meaning.**

Enhances readability by ensuring all concepts are represented uniquely. Reduce chance of error by side effects. Don't fall into the temptation of reusing an existing variable in a new context.

44. **Class variables should never be declared public.**

The concept of Java information hiding and encapsulation is violated by public variables. Use private variables and access functions instead. One exception to this rule is when the class is essentially a data structure, with no behavior (equivalent to a C++ `struct`). In this case it is appropriate to make the class' instance variables public.

45. **Arrays should be declared with their brackets next to the type.**

```
double[] vertex; // NOT: double vertex[];
int[] count; // NOT: int count[];

public static void main(String[] arguments)

public double[] computeVertex()
```

The reason for is twofold. First, the *array-ness* is a feature of the class, not the variable. Second, when returning an array from a method, it is not possible to have the brackets with other than the type (as shown in the last example).

46. **Variables should be kept alive for as short a time as possible.**

Keeping the operations on a variable within a small scope makes it easier to control its effects and potential side effects.

Loops

47. **Only loop control statements must be included in the `for()` construct.**

```
sum = 0;                    // NOT: for (i = 0, sum = 0; i < 100; i++)
for (i = 0; i < 100; i++)   //        sum += value[i];
  sum += value[i];
```

Increase maintainability and readability. Make a clear distinction of what *controls* and what is *contained* in the loop.

48. **Loop variables should be initialized immediately before the loop.**

```
boolean isDone = false;   // NOT: boolean isDone = false;
while (!isDone) {         //      :
  :                       //      while (!isDone) {
}                         //        :
                          //      }
```

Increase maintainability and readability.

49. **The use of `do-while` loops can be avoided.**

`do-while` loops are less readable than ordinary `while` loops and `for` loops since the conditional is at the bottom of the loop. The reader must scan the entire loop in order to understand its scope.

In addition, `do-while` loops are not needed. Any `do-while` loop can easily be rewritten into a `while` loop or a `for` loop. Reducing the number of different constructs used enhance readability.

Conditionals

50. **Complex conditional expressions must be avoided. Introduce temporary boolean variables instead.**

```
bool isFinished = (elementNo < 0) || (elementNo > maxElement);
bool isRepeatedEntry = elementNo == lastElement;
if (isFinished || isRepeatedEntry) {
  :
}

// NOT:
if ((elementNo < 0) || (elementNo > maxElement)||
     elementNo == lastElement) {
  :
}
```

By assigning boolean variables to expressions, the program gets automatic documentation. The construction will be easier to read, debug and maintain.

51. **The *happy case* should be put in the `if`-part and the exception in the `else`-part of an if statement.**

```
boolean isOk = readFile(fileName);
if (isOk) {
```

```
    :
  }
  else {
    :
  }
```

Makes sure that the exceptions does not obscure the normal path of execution. This is important for both the readability and performance.

52. **The conditional should be put on a separate line.**

```
if (isDone)    // NOT: if (isDone) doCleanup();
  doCleanup();
```

This is for readability, but also for debugging: When writing on a single line, it is not immediately apparent whether the test is really true or not.

53. **Executable statements in conditionals must be avoided.**

```
InputStream stream = File.open(fileName, "w");
if (stream != null) {
  :
}

// NOT:
if (File.open(fileName, "w") != null)) {
  :
}
```

Conditionals with executable statements are simply very difficult to read. This is especially true for programmers new to Java.

Miscellaneous

54. **The use of magic numbers in the code should be avoided. Numbers other than `0` and `1` can be considered declared as named constants instead.**

```
private static final int  TEAM_SIZE = 11;
:
Player[] players = new Player[TEAM_SIZE]; // NOT: Player[] players = new Player[11]
```

If the number does not have an obvious meaning by itself, the readability is enhanced by introducing a named constant instead.

55. **Floating point constants should always be written with decimal point and at least one decimal.**

```
double total = 0.0;    // NOT:  double total = 0;
double speed = 3.0e8;  // NOT:  double speed = 3e8;

double sum;
:
sum = (a + b) * 10.0;
```

This emphasize the different nature of integer and floating point numbers. Mathematically the two model completely different and non-compatible phenomenons.

Also, as in the last example above, it emphasize the type of the assigned variable (`sum`) at a point in the code where this might not be evident.

56. **Floating point constants should always be written with a digit before the decimal point.**

```
double total = 0.5;  // NOT:  double total = .5;
```

The number and expression system in Java is borrowed from mathematics and one should adhere to mathematical conventions for syntax wherever possible. Also, `0.5` is a lot more readable than `.5`; There is no way it can be mixed with the integer `5`.

57. **Type conversions must always be done explicitly. Never rely on implicit type conversion.**

```
floatValue = (int) intValue; // NOT: floatValue = intValue;
```

By this, the programmer indicates that he is aware of the different types involved and that the mix is intentional.

Layout

58. **Basic indentation should be 2.**

```
for (i = 0; i < nElements; i++)
  a[i] = 0;
```

Indentation is used to emphasize the logical structure of the code. Indentation of 1 is to small to achieve this. Indentation larger than 4 makes deeply nested code difficult to read and increase the chance that the lines must be split. Choosing between indentation of 2, 3 and 4; 2 and 4 are the more common, and 2 chosen to reduce the chance of splitting code lines.

59. **Block layout should be as illustrated in example 1 below (recommended) or example 2, and must not be as shown in example 3. Class, Interface and method blocks should use the block layout of example 2.**

```
// Recommended
while (!done) {
  doSomething();
  done = moreToDo();
}

// Viable alternative
while (!done)
{
  doSomething();
  done = moreToDo();
}

// Don't do this
while (!done)
  {
    doSomething();
    done = moreToDo();
  }
```

Example 3 introduce an extra indentation level which doesn't emphasize the logical structure of the code as clearly as example 1 and 2.

60. **The `class` and `interface` declarations should have the following form:**

```
class Rectangle extends Shape
  implements Cloneable, Serializable
{
  ...
}
```

This follows from the general block rule above. Note that it is common in the Java developer community to have the opening bracket at the end of the line of the class keyword. This is not recommended.

61. **Method definitions should have the following form:**

```
public void someMethod()
  throws SomeException
{
  ...
}
```

See comment on class statements above.

62. **The `if-else` class of statements should have the following form:**

```
if (condition) {
  statements;
}

if (condition) {
  statements;
}
else {
   statements;
}

if (condition) {
  statements;
}
else if (condition) {
  statements;
}
else {
  statements;
}
```

This follows partly from the general block rule above. However, it might be discussed if an else clause should be on the same line as the closing bracket of the previous if or else clause:

```
if (condition) {
  statements;
} else {
  statements;
}
```

The chosen approach is considered better in the way that each part of the if-else statement is written on separate lines of the file. This should make it easier to manipulate the statement, for instance when moving `else` clauses around.

63. **The `for` statement should have the following form:**

```
for (initialization; condition; update) {
  statements;
}
```

This follows from the general block rule above.

64. **An empty `for` statement should have the following form:**

```
for (initialization; condition; update)
  ;
```

This emphasize the fact that the for statement is empty and it makes it obvious for the reader that this is intentional.

65. **The `while` statement should have the following form:**

```
while (condition) {
  statements;
}
```

This follows from the general block rule above.

66. **The `do-while` statement should have the following form:**

```
do {
  statements;
} while (condition);
```

This follows from the general block rule above.

67. **The `switch` statement should have the following form:**

```
switch (condition) {
  case ABC :
    statements;
    // Fallthrough

  case DEF :
    statements;
    break;

  case XYZ :
    statements;
    break;

  default :
    statements;
    break;
}
```

This differs slightly from the Java community recommendation both in indentation and spacing. In particular, each `case` keyword is indented relative to the switch statement as a whole. This makes the entire switch statement stand out. Note also the extra space before the `:` character. The explicit *Fallthrough* comment should be included whenever there is a `case` without a `break`. Leaving the `break` out is a common error, and it must be made clear that its omission is intentional.

68. **The `try-catch` statement should have the following form:**

```
try {
  statements;
}
catch (Exception exception) {
  statements;
}

try {
  statements;
}
catch (Exception exception) {
  statements;
}
finally {
  statements;
}
```

This follows partly from the general block rule above. This form differs from the Java community recommendation in the same way as the `if-else` statement described above.

69. **Single statement `if-else`, `for` or `while` statements can be written without brackets.**

```
if (condition)
  statement;

while (condition)
  statement;

for (initialization; condition; update)
  statement;
```

It is a common recommendation (Java community included) that brackets should always be used in all these cases. However, brackets are in general a language construct that groups multiple statements. Brackets are per definition superfluous on a single statement. A common argument against this syntax is that the code will break if an additional statement is added without also adding the brackets. In general however, code should never be written to accommodate for changes that *might* arise.

70. **Whitespace should be included to improve readabilty**

```
a = (b + c) * d; // NOT: a=(b+c)*d

while (true) {   // NOT: while(true){
  ...

doSomething(a, b, c, d);  // NOT: doSomething(a,b,c,d);

case 100 :  // NOT: case 100:

for (i = 0; i < 10; i++) {  // NOT: for(i=0;i<10;i++){
  ...
```

In general:

- Operators should be surrounded by a space character.
- Java reserved words should be followed by a space.
- Commas should be followed by a space.
- Colons should be surrounded by space.
- Semicolons in for statements should be followed by a space.

Makes the individual components of the statements stand out and enhances readability.

It is difficult to give a complete list of the suggested use of whitespace in Java code. The examples above however should give a general idea of the intentions.

71. **Logical units within a block should be separated by one blank line.**

```
// Create a new identity matrix
Matrix4x4 matrix = new Matrix4x4();

// Precompute angles for efficiency
double cosAngle = Math.cos(angle);
double sinAngle = Math.sin(angle);

// Specify matrix as a rotation transformation
matrix.setElement(1, 1,  cosAngle);
matrix.setElement(1, 2,  sinAngle);
matrix.setElement(2, 1, -sinAngle);
matrix.setElement(2, 2,  cosAngle);

// Apply rotation
transformation.multiply(matrix);
```

Enhances readability by introducing whitespace between logical units. Each block is often introduced by a comment as indicated in the example above.

72. **Statements can be aligned wherever this enhances readability.**

```
if      (a == lowValue)    compueSomething();
else if (a == mediumValue) computeSomethingElse();
else if (a == highValue)   computeSomethingElseYet();

value = (potential        * oilDensity)   / constant1 +
        (depth            * waterDensity) / constant2 +
        (zCoordinateValue * gasDensity)   / constant3;

minPosition     = computeDistance(min,     x, y, z);
averagePosition = computeDistance(average, x, y, z);

switch (phase) {
  case PHASE_OIL   : text = "Oil";   break;
  case PHASE_WATER : text = "Water"; break;
  case PHASE_GAS   : text = "Gas";   break;
}
```

There are a number of places in the code where whitespace can be included to enhance readability even if this violates common guidelines. Many of these cases have to do with code alignment. General guidelines on code alignment are difficult to give, but the examples above should give some general hints. In short, any construction that enhances readability should be allowed.

73. **Tricky code should not be commented, but rewritten.**

In general, the use of comments should be minimized by making the code self-documenting by appropriate name choices and an explicit logical structure.

74. **All comments should be written in English.**

Commenting in English simplifies global collaboration, ensure consistency between systems, simplify onboarding process, ensure maintainability and maintains technical jargon.

75. **Javadoc comments should have the following form:**

```
/**
 * Return lateral location of the specified position.
 * If the position is unset, NaN is returned.
 *
 * @param x     X coordinate of position.
 * @param y     Y coordinate of position.
 * @param zone  Zone of position.
 * @return      Lateral location.
 * @throws IllegalArgumentException  If zone <= 0.
 */
public double computeLocation(double x, double y, int zone)
{
  ...
}
```

A readable form is important because this type of documentation is typically read more often inside the code than as processed text.

Note in particular:

set
- The opening `/**` on a separate line
- Subsequent `*` is aligned with the first one
- Space after each `*`
- Empty line between description and parameter section.
- Alignment of parameter descriptions.
- Punctuation behind each parameter description.
- No blank line between the documentation block and the method/class.

76. **There should be a space after the comment identifier.**

```
// This is a comment    NOT: //This is a comment

/**                     NOT: /**
 * This is a javadoc          *This is a javadoc
 * comment                    *comment
 */                          */
```

Improves readability by making the text stand out.

77. **`//` should be used for all non-JavaDoc comments, including multi-line comments.**

```
// Comment spanning
// more than one line.
```

Since multilevel Java commenting is not supported, using `//` comments ensure that it is always possible to comment out entire sections of a file using `/* */` for debugging purposes etc.

78. **Comments should be indented relative to their position in the code.**

```
while (true) {
  // Do something
  something();
}                             }

// NOT:
while (true) {
// Do something
  something();
}
```

This is to avoid that the comments break the logical structure of the program.

79. **All public classes and public and protected functions within public classes should be documented using the Java documentation (Javadoc) conventions.**

This makes it easy to keep up-to-date online code documentation.

It is a good practice to always document package private and private content in the same way.

80. **All class members should be properly documented.**

```java
public final class License
{
  /** The license holder, i.e. the licensee. Non-null. */
  private final String licensee_;

  /** License issuer. Non-null. */
  private final String issuer_;

  /** Name of product being licensed. Non-null. */
  private final String product_;

  /** ID of product the license is valid for. Non-null. */
  private final String productId_;

  /** Last date the license is valid. Or null if it never expires. */
  private final Date expireDate_;

  :
}
```

Provide proper documentation of the intent for all class members. Should clearly identify value range and if it can be null and in case, what this means.

If synchronization is an issue the members should identify by which lock it is protected.

81. **Overridden methods must be tagged by `@Override` and instructed to inherit documentation from their base class.**

```java
/** {@inheritDoc} */
@Override
public String toString()
{
  :
}
```

Ensures correct API documentation and will cause compilation error if the method signature is wrong. Do this also for interface methods.

There should be no reason to provide specific documentation for such methods. If the base class documentation is not sufficient it is a clear indication that the logic in the override should be in a different method.

Best practices

82. **Classes should be `final` unless they are explicitly designed for inheritance.**

```
public final class Location
{
  :
}
```

Very few classes are inherited, and creating proper base classes takes serious effort. Making a class `final` indicates for the client that the class is not designed for or meant to be inherited.

83. **Accessibility of classes and members should be minimized.**

If possible:

- Prefer package classes to public classes.
- Prefer final classes to non-final classes.
- Prefer private inner classes to package classes if the class is local to one other class only.
- Prefer private static inner classes to private non-static inner classes.
- Prefer package methods to public methods.
- Prefer private methods to package methods.
- Prefer static private methods to private methods if the class members are not referenced, or it is natural to pass these as methods arguments.
- Prefer private members to protected members.

84. **Class members should whenever possible be immutable (final) with its state established at declaration or in the constructor.**

```
public final class Node
{
  private final List<Node> children_ = new ArrayList<>();

  :
}


public final class Location
{
  private final double latitude_;

  private final double longitude_;

  public Location(double latitude, double longitude)
  {
    latitude_ = latitude;
    longitude_ = longitude;
    :
```

```
    }

    :
  }
```

Keeps the class state easier to control, reduces the need for setter methods and simplifies threading.

85. **Mutable members must not be exposed through a public API without explicit notice.**

```
public List<Point> getPoints()
{
  return Collections.unmodifiableList(points_);
}

// NOT:
public List<Point> getPoints()
{
  return points_;
}



public Date getDate()
{
  return date_ != null ? new Date(date_.getTime()) : null;
}

// NOT:
public Date getDate()
{
  return date_;
}



public double[] getCoordinates()
{
  return Arrays.copyOf(coordinates_, coordinates_.length);
}

// NOT:
public double[] getCoordinates()
{
  return coordinates_;
}
```

Providing access to mutable members violates encapsulation and can cause defects that are very hard to find.

86. **Mutable arguments must be copied when received by public methods.**

```
public void addValues(List<Value> values)
{
  values_.addAll(values);
}

public void setDate(Date date)
{
  date_ = date != null ? new Date(date.getTime()) : null;
}

public void setCoordinates(double[] coordinates)
{
  coordinates_ = Arrays.copyOf(coordinates, coordinates.length);
}
```

Enforces encapsulation.

87. **Valid argument range of public methods must be documented and validated with `IllegalArgumentException`.**

```
/**
 * Specify the length of this line segment.
 *
 * @param length  Length to set. [0.0, >.
 * @throws IllegalArgumentException  If length < 0.
 */
public void setLength(double length)
{
  if (length < 0.0)
    throw new IllegalArgumentException("Invalid length: " + length);

  :
}


/**
 * Create a new Type instance.
 *
 * @param id           Type ID. Non-null.
 * @param displayName  Display name of the type. Null if unspecified.
 */
```

```
public Type(String id, String displayName)
{
  if (id == null)
    throw new IllegalArgumentException("id cannot be null");

  :
}
```

Makes it clear for the client what the API accepts and not. The exception is to break the program flow as soon as possible after an invalid call.

88. **Valid argument range of private methods must be documented and validated with `assert`.**

```
/**
 * Specify the length of this line segment.
 *
 * @param length  Length to set. [0.0, >.
 * @throws IllegalArgumentException  If length < 0.0.
 */
private void setLength(double length)
{
  assert length >= 0.0 : "Invalid length: " + length);
  :
}


/**
 * Add a new element to this instance.
 *
 * @param element  Element to add. Non-null.
 */
private void addElement(Element element)
{
  assert element != null : "element cannot be null";
  :
}
```

Similar to the public counterpart, but private methods that are called with illegal arguments identifies programmer errors and there is no reason the problem should be passed up to the calling class through an exception.

89. **The possible value range of return values must be properly documented.**

```
/**
 * Return the number of values in this collection.
 *
 * @return  Number of values in this collection. [0,>.
 */
public int getNValues()
{
  :
}

/**
 * Return the children of this node.
 *
 * @return  The children of this node. Never null.
 */
private List<Node> getChildren()
{
  :
}

/**
 * Return the description of this entry.
 *
 * @return  Description of this entry. Null if none provided.
 */
private String getDescription()
{
  :
}
```

Ensures correct API usage.

90. **The `equals()` method must be overridden if different objects should report as equal according to the logical model.**

```
class Task
{
  :

  /** {@inheritDoc} */
  @Override
  public boolean equals(Object object)
  {
    if (object == null)
      return false;

    if (object == this)
      return true;
```

```
    if (!(object instanceof Task))
      return false;

    Task task = (Task) object;

    // Compare every member. Beware of null checking.
    if (!task.member1_.equals(member1_))
      return false;

    :

    return true;
  }
}
```

The base class `equals()` method will compare object pointers and this is not sufficient for comparing the internal state of objects.

The general format of the overridden `equals()` method should follow the template above.

Always consider the alternative approach of implementing `equals()` which is to ensure that equal objects always are the *same* instance. This can be achieved by using factories that manage the pool of created objects.

91. The `hashCode()` method must always be overridden when `equals()` is.

```
class Task
{
  :

  /** {@inheritDoc} */
  @Override
  public int hashCode()
  {
    return Objects.hash(member1_, member2_, ...);
  }
}
```

When instances are used as keys in hashed collections it is essential that objects that reports as equal have identical hash code. This will in general not happen if `equals()` is overridden and `hashCode()` is not.

The `hashCode()` method must be fast and should create its integer code from the hash codes of its *immutable members* and should ensure sufficient distribution of codes from the possible keys. This may involve convoluted mathematics including prime number factors etc. For the general

client it is advised to simply use the logic provided by the JDK in the `Objects.hash(...)` method as shown above.

Remember: The hash code *must never* be based on mutable members!

92. **Objects should be referred to by their simplest possible interface.**

```
// NOT: The ArrayList class is most probably not needed here
ArrayList<Point> points = new ArrayList<>();

// This is better
List<Point> points = new ArrayList<>();

// Even better if list indexing is not needed
Collection<Point> points = new ArrayList<>();

// Even better if only iteration is needed
Iterable<Point> points = new ArrayList<>();
```

Improve flexibility as the implementing type can be changed with less impact.

93. **Utility classes (classes with static methods only) should have a private constructor.**

```
public final class HardwareId
{
  /**
   * Private constructor to prevent client instantiation.
   */
  private HardwareId()
  {
    assert false : "This constructor should never be called";
  }

  :
}
```

If the constructor is not created explicitly, a default public one is generated. This is not wanted as instances of such classes makes no sense.

94. **Checked exceptions must be properly handled.**

There are four ways of handling exceptions thrown by a call within a method:

1. Pass the exception on to the client. Do this if the exception cannot be adequately handled at this level, or the information provided by the exception is *better* handled by the client code:

```
void method()
  throws SomeException
{
  :
}
```

2. Catch the exception and pass on a *different* checked exception. Do this if the condition should be handled by the client code, but at a higher level of abstraction:

```
try {
  :
}
catch (SomeException exception) {
  throw new OtherException(..., exception);
}
```

3. Handle the exception, typically by closing resources and logging:

```
try {
  :
}
catch (SomeException exception) {
  :
  logger_.log(Level.WARNING, "Operation failed ... ", exception);
}
```

4. Ignore the exception. Do this only if it is *certain* that the exception will not be thrown, and document the fact with assert:

```
try {
  :
}
catch (SomeException exception) {
  assert false : "This will never happen";
}
```

95. **The `toString()` method should always be overridden.**

```
/** {@inheritDoc} */
@Override
public String toString()
{
  return name_ + " " + value_ + ...;
}
```

This is recommended by the Object base class: *"It is recommended that all subclasses override the toString method. The string should be a concise but informative representation that is easy for humans to read."*

Providing a good `toString` implementation simply makes classes much more pleasant to use, such as during debugging or with printouts.

If practical, a string representation of *all* the members of a class should be included in the `toString()`.

96. **Code must never rely on the `toString()` method.**

According to the previous advice, `toString()` should always be overridden.

However convenient, the code must never *rely* on the exact format of this, and if a specific string representation of a class content is used in the business logic, this logic must be provided by a different method than `toString()`, like `toXml()` or `toJson()` etc.

The reason is flexibility. `toString()` should contain a scratch representation of the class content convenient for debugging or logging, and it should be possible to modify (for instance when the class changes) without any impact on the surrounding code.

97. **Methods that returns collections should never return `null`. Return an empty collection instead.**

```
/**
 * Return the points of this shape.
 *
 * @return  The points of this shape. Never null.
 */
List<Point> getPoints()
{
  ...
}
```

This simply makes code much more robust and client code much easier to write. Methods that returns `null` for collections (like `java.io.File.listFiles()`) is a production disaster waiting to occur.

According to the documentation rule above, always document the fact that `null` is never returned.

98. **Level of nesting should be kept as low as possible.**

```
// NOT: Don't do this
for (int i = 0; i < lines.length; i++) {
  if (!line.isEmpty() && !line.startsWith("#")) {
    String tokens = line.split("=");
```

```
    if (tokens.size == 2) {
      String key = tokens[0];
      if (!key.endsWith("_")) {
        //
        // Handle nominal case here
        //
      }
    }
  }
}

// Do this instead
for (int i = 0; i < lines.length; i++) {
  if (line.isEmpty() || line.startsWith("#")
    continue;

  String tokens = line.split("=");
  if (tokens.size != 2)
   continue;

  String key = tokens[0];
  if (key.endsWith("_"))
    continue;

  //
  // Handle nominal case here
  //
}



// NOT: Never use else after returning from an if-clause
int getMagnitude(double v)
{
  if (Double.isNaN(v)) {
    return -1;
  }
  else {
    //
    // Handle nominal case here
    //
  }
}

// Do this instead
int getMagnitude(double v)
{
  if (Double.isNaN(v))
    return -1;

  //
```

```
    // Handle nominal case here
    //
  }



  // NOT: Don't use if-else if the ternary operator can be used instead
  String text;
  if (Double.isNaN(value))
    text = "";
  else
    text = value.toString();

  // Do this instead
  String text = Double.isNaN(value) ? "" : value.toString();
```

In general, use `continue`, `break` and `return` wherever this simplifies the code structure, prohibits deep nesting and improves readability.

It is often argued that `continue` and `break` are variants of the infamous `goto` statement and that they therefore must be avoided. Likewise, it is argued that methods should contain a single point of return only. The examples above indicates that readability is greatly improved if instead the level of nesting is reduced as much as possible.

Background

The guidelines provided above are intended to improve the readability of code and make it consistent across the wide spectrum of Java projects.

A style guide is about *consistency*. Consistency with this style guide is important. Consistency within a project is more important. Consistency within one package, class or method is the most important.

However, know when to be inconsistent - sometimes style guide recommendations just aren't applicable. When in doubt, use the best of judgment. Look at example code and decide what looks best.

The recommendations are grouped by topic and each recommendation is numbered to make it easier to refer to during reviews. In the guideline sections the terms *must*, *should* and *can* have special meaning. A *must* requirement must be followed, a *should* is a strong recommendation, and a *can* is a general guideline.

Layout for the recommendations is as follows:

> **Guideline short description.**
>
> ```
> Example if applicable
> ```
>
> Rationale, motivation, background and additional information.

The GeoSoft Java Programming Style Guidelines was first published in 1999, but it has been revised and extended a number of times. The Last revison is from 2024.

This work is licensed under the Creative Commons Attribution 4.0 International license.

Copyright © 2024 - Geotechnical Software Services - All rights reserved
Designed and created by GeoSoft - *Simplicity is the ultimate sophistication*